

DATA ENGINEERING INTERVIEW

Technical Assessment: Football Analytics Pipeline

Build a modular, production-grade ETL pipeline using the Kaggle Football Dataset. Demonstrate mastery of Medallion Architecture, Data Quality, Orchestration, and Containerised Deployment.

KEY REQUIREMENTS AT A GLANCE

Medallion Architecture	Star Schema Data Modelling	SCD Type 2 & Historical Loads	Data Quality Checks
Airflow Orchestration	Unit Testing (pytest)	Dockerised Execution	Modular & Config-driven

1. Objective

Build a modular, production-grade ETL data pipeline using the Kaggle Football Dataset. Design a multi-layered transformation architecture (Bronze → Silver → Gold) with data quality enforcement, orchestration, containerised deployment, and comprehensive testing.

Dataset

Kaggle Football Dataset — Player Scores

<https://www.kaggle.com/datasets/davidcariboo/player-scores/data>

Focus on the following tables (extract full tables in the ETL process):

Table	Description
players	Player master records
games	Match records
appearances	Player appearances per game
clubs	Club master records
competitions	Competition master records
player_valuations	Historical player market values

2. Tech Stack Requirements

Requirement	Specification
Language	Python
Tooling Preference	Open-source tools wherever possible
Execution Environment	Must run on a standard laptop
Reproducibility	Full setup reproducible via Docker

3. Medallion Architecture

The pipeline must follow a strict Bronze → Silver → Gold layered architecture:

Layer	Purpose & Responsibilities
 Bronze (Raw)	Raw ingestion with minimal transformations. Enforce schema. Store as .parquet files.
 Silver (Curated)	Cleaned, deduplicated, and type-cast data. Structured into a Star Schema.
 Gold (Aggregated)	Aggregated business-ready tables: Player Performance, Club Stats, Player Valuation Trend.

Gold Layer — Example Output Tables

Player Performance Table	Club Performance Table	Player Valuation Trend
player_id	club_id	player_id
season / year	matches_played	date
total_goals	wins / losses	market_value
total_assists	goals_scored	rolling_average
total_minutes		

4. Data Modeling — Star Schema

Table Type	Tables
Fact Tables	fact_appearances, fact_games

Dimension Tables	dim_players, dim_clubs, dim_competitions, dim_date
------------------	--

5. Incremental & Historical Loads

A. Delta / Incremental Load

- Simulate new data arrival (configurable frequency — default: hourly)
- Process only new or changed records
- Example triggers:**
 - New games added
 - Updated player valuations

Note: Generate a second snapshot (e.g., a `data_day2/` folder) that represents "new data arriving" with:

- A few **new rows** added (e.g. 2 to 3 new game records with future dates, new player appearances)
- A few **updated rows** (e.g., player market value changed in `player_valuations.csv`, or a player's `current_club_id` changed in `players.csv` this is ideal for testing SCD Type 2)
- Optionally, a **deleted row** (soft delete simulation, can add a `is_deleted` flag)

B. Historical Load — Slowly Changing Dimensions (SCD)

- Implement SCD Type 2 for players and/or valuations
- Track the following SCD fields:**
 - `effective_date`
 - `end_date`
 - `is_current`

6. Data Quality (DQ)

Required Checks

Check Type	Rule
Null Checks	player_id and game_id must not be null
Referential Integrity	player_id must exist in the players table
Valid Ranges	minutes_played ≥ 0 goals ≥ 0

Bonus Checks

- Duplicate detection

- Schema drift handling

Output

- DQ report / structured logs per pipeline run

7. Transformations

The pipeline must perform the following transformations (at minimum):

Transformation	Description
Normalise Positions	Standardise player position labels to a defined taxonomy
Standardise Country Names	Map raw country strings to ISO standard names
Match Outcome	Derive win / loss / draw from score data
Season Derivation	Extract season year from match date

8. Pandas vs. Spark — Decision Required

⚠ You must choose EITHER Pandas OR PySpark (Spark) for this assignment.

Justify your choice clearly in your README and/or documentation.

Justification Point	What to Address
Choice Rationale	Why Pandas or Spark is appropriate for this dataset
Scale Threshold	At what dataset size would you switch from Pandas to Spark?
Modularity (Bonus/Nice to have)	Architecture should allow swapping Pandas ↔ Spark with minimal code changes

9. Airflow Orchestration

DAG Pipeline Flow

ingestion → Data Quality (DQ) → Silver Transformation → Gold Aggregation

DAG Requirements

- Accepts new feed every hour (configurable frequency)
- Include task retries on failure
- Structured logging for all tasks
- Idempotent execution — re-running must not cause duplicate data

10. Unit Testing

Use pytest or unittest. Tests must cover all critical pipeline components.

Test Category	What to Test
Transformations	Validate derived fields, normalisations, and type casts
Data Quality Checks	Verify DQ rules catch invalid records correctly
Incremental Logic	Ensure only new/changed records are processed
SCD Logic	Validate effective_date, end_date, and is_current tracking

11. Docker & Airflow Integration

Docker Compose Setup

Your docker-compose.yml must spin up all required services:

Service	Role
Airflow Webserver	UI available at localhost:8080
Airflow Scheduler	Triggers and monitors DAG runs
Pipeline Container	Executes the ETL pipeline end-to-end

Expected Execution

```
docker-compose up
```

- ✓ Airflow UI accessible at http://localhost:8080
- ✓ DAG visible and manually runnable from the UI
- ✓ Full pipeline executes end-to-end without manual intervention

12. Code Quality & Modularity

Principle	Requirement
Modular Structure	Clear separation of concerns — ingestion, transform, DQ, load
Config-Driven	No hard-coded values; use configuration files (YAML / .env)
Extensibility	Adding a new dataset should require minimal code changes
Plug-and-Play	Pandas ↔ Spark swap achievable with minimal refactoring

13. Deliverables

All deliverables must be submitted via a public GitHub repository.

Deliverable	Details
Source Code	Full, runnable Python codebase in a public GitHub repository
Docker Configuration	Dockerfile + docker-compose.yml for containerised execution
README	Detailed instructions to clone, configure, and run the solution
Airflow DAG	Well-documented DAG with all required tasks
Unit Tests	pytest / unittest test suite covering all critical components

Suggested project structure:

```
project/
├── dags/
│   └── football_pipeline_dag.py
├── src/
│   ├── ingestion/
│   ├── bronze/
│   ├── silver/
│   ├── gold/
│   ├── dq/
│   ├── models/
│   └── utils/
├── tests/
│   ├── test_ingestion.py
│   ├── test_transformations.py
│   ├── test_dq.py
│   └── test_incremental.py
├── configs/
│   └── config.yaml
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── README.md
```

14. Optional / Nice-to-Have

Feature	Description
Reporting Tool	Integrate an open-source reporting tool (e.g., Metabase, Superset) for statistics and insights
PII Anonymisation	Anonymise personally identifiable fields in the dataset before storage

Good luck! We look forward to reviewing your solution.

Focus on clean code, sound engineering decisions, and clear documentation.